PRACTICAL WEEK 03

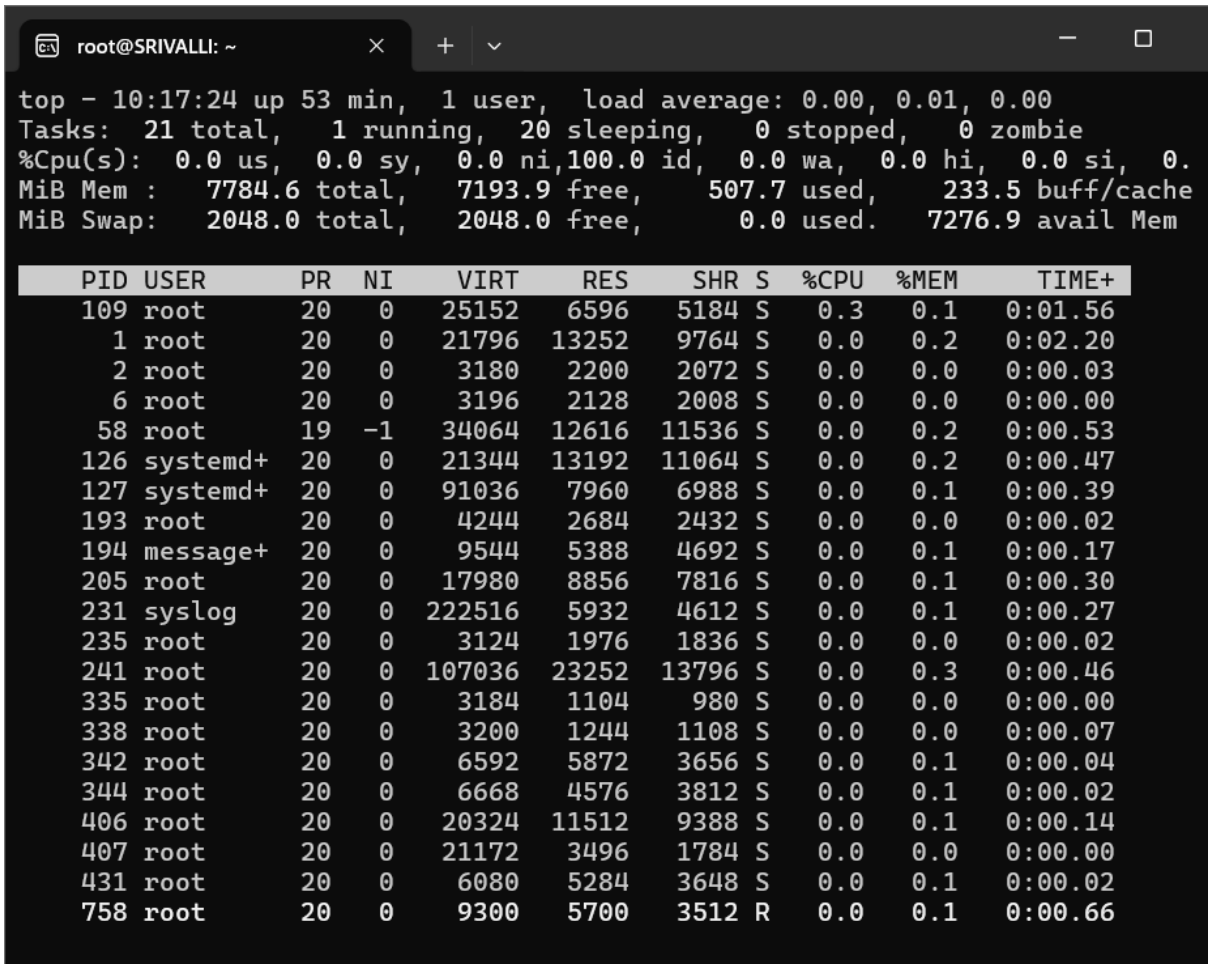
1.Develop a C program using frok() that creates a parent and child process.Displays the Process ID(PID0,Parent Process ID(PPID),and process states at different states at different stages of execution.

```
root@SRIVALLI:~# nano fork_process.c
root@SRIVALLI:~# gcc fork_process.c -o fork_process
root@SRIVALLI:~# ./fork_process

Parent Process

Parent PID: 522
Child Process
Child PID: 523
Parent is waiting for child...
Child PID: 523
Parent PID: 522
Child is running...
Child process completed.
Child process has completed.
Parent process completed.
```

Report:

In this task, I used the fork() system call in C to create a parent process and a child process. The program displayed the PID of the parent and child processes and also showed the Parent Process ID (PPID). The wait() function was used so that the parent process waits for the child process to finish. This helped me understand how processes are created and how parent and child processes work in Linux.

2.Design an experiment to observe process state transitions(Ready,Running,Waiting,Terminated)using Linux monitoring tools such as ps,top,and/proc.Document the observations.



Report: In this task, I used the fork() system call in C to create a parent process and a child process. The program displayed the PID of the parent and child processes and also showed the Parent Process ID (PPID). The wait() function was used so that the parent process waits for the child process to finish. This helped me understand how processes are created and how parent and child processes work in Linux.